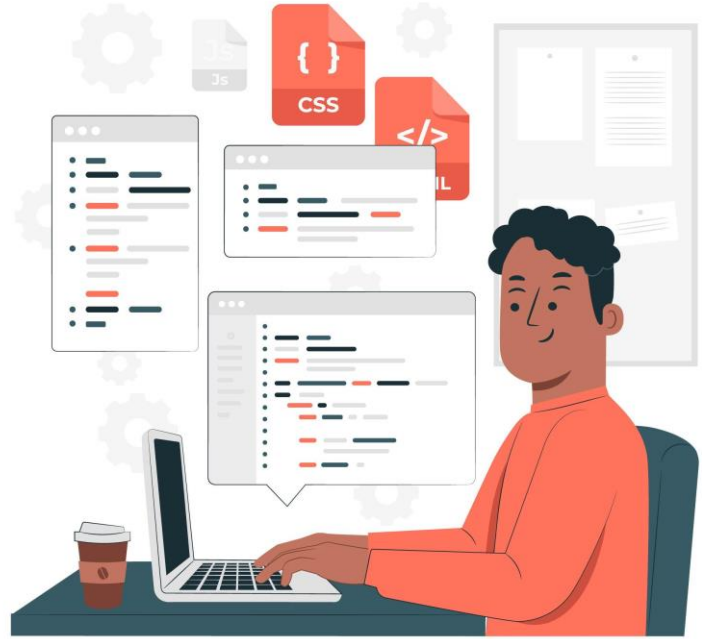


JWT Auth





KEMNAKER

Outline

- JWT
- Penerapan JWT Auth di Express.js

JWT

Apa itu JWT?

JWT, atau **JSON Web Token**, adalah standar terbuka (RFC 7519) yang **mendefinisikan format token yang ringan** dan dapat dibaca oleh mesin. JWT biasanya digunakan untuk **mentransfer klaim** antara pihak-pihak dalam cara yang dapat diverifikasi. Informasi ini dapat diverifikasi dan dipercayai karena ditandatangani secara digital.

JWT dapat ditandatangani menggunakan algoritma HMAC (Hash-Based Message Authentication Code) atau menggunakan pasangan kunci kriptografis (RSA atau ECDSA).

Sebuah token JWT terdiri dari tiga bagian terenkripsi dalam bentuk Base64 URL, yang masing-masing disebut sebagai **header, payload, dan signature**

<https://jwt.io/>

Apa itu JWT?

JWT (JSON Web Token) adalah sebuah **standar** untuk melakukan **otentikasi dan pertukaran informasi yang aman** antar sistem dalam bentuk **token** (teks string yang terencode).

JWT biasanya digunakan pada **web application** atau **API authentication** supaya server bisa mengenali pengguna tanpa harus menyimpan session di server.

Di **Express.js (Node.js)**, JWT bisa digunakan untuk **authentication & authorization** dengan bantuan library, misalnya:

- [jsonwebtoken](#) → library untuk membuat & memverifikasi JWT.
- [express-jwt](#) → middleware untuk proteksi route pakai JWT.

Struktur JWT

JWT terdiri dari **3 bagian** yang dipisahkan oleh tanda titik (.):
header.payload.signature

- **Header** → info enkripsi
- **Payload** → data user + waktu expired
- **Signature** → validasi (dibuat dari header + payload + secret)

Struktur JWT

JWT terdiri dari **3 bagian** yang dipisahkan oleh tanda titik (.):
header.payload.signature

1. Header

Berisi tipe token (JWT) dan algoritma enkripsi yang digunakan, misalnya HS256.

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```


Struktur JWT

JWT terdiri dari **3 bagian** yang dipisahkan oleh tanda titik (.):
header.payload.signature

2. Payload

Berisi data (claims) yang ingin disimpan/dikirim di dalam token, misalnya userId, role, atau informasi lain. Contoh :

```
{  
  "sub": "1234567890",  
  "name": "John Doe",  
  "admin": true  
}
```

```
{  
  "id": 1,  
  "email": "user@email.com",  
  "iat": 1694682000,  
  "exp": 1694685600  
}
```

- iat = issued at (kapan token dibuat)
- exp = expired (kapan token kadaluarsa)
- Sisanya = data yang kita pilih untuk ditaruh (contohnya id, email, role, dll).

Tips

Jangan simpan data sensitif (password, dll) di payload JWT karena payload mudah di-decode

Struktur JWT

JWT terdiri dari **3 bagian** yang dipisahkan oleh tanda titik (.):
header.payload.signature

3. Signature

Bagian ini **adalah hasil *hash dari header + payload** menggunakan **jwt secret key**, supaya token tidak bisa dipalsukan.

Untuk membuat bagian signature, Anda harus mengambil header yang dikodekan, payload yang dikodekan, sebuah JWT SECRET, algoritma yang ditentukan dalam header, dan menandatanganinya. Misalnya, jika Anda ingin menggunakan algoritma HMAC SHA256, tanda tangan akan dibuat dengan cara berikut:

```
HMACSHA256(  
    base64UrlEncode(header) + "." +  
    base64UrlEncode(payload),  
    secret)
```

*Hash ?

Hashing adalah proses matematika satu arah yang **mengubah data input** dengan ukuran berapa pun **menjadi string karakter dengan ukuran tetap**, yang disebut nilai hash (atau hash).

Proses ini berfungsi seperti sidik jari digital: masukan yang sama selalu menghasilkan hash yang sama, tetapi masukan yang sedikit berbeda akan menghasilkan hash yang sangat berbeda.

Hashing digunakan untuk memverifikasi integritas data (memastikan tidak ada yang diubah), menyimpan kata sandi dengan aman, dan dalam teknologi blockchain serta tanda tangan digital.

Algorithm

HS256

Struktur JWT

JWT terdiri dari **3 bagian** yang dipisahkan oleh tanda titik (.):
header.payload.signature

Encoded

```
eyJhbGciOiJIUzI1NiIsI  
nR5cCI6IkpXVCJ9.eyJz  
dWUiOiIxMjM0NTY3ODkwI  
iwibmFtZSI6IkpvaG4gRG  
9lIiwiaWF0IjoxNTE2MjM  
MDIyfQ.SflKxwRJSMeKKF  
2QT4fwpMeJf36P0k6yJV_  
adQssw5c
```

Decoded

HEADER:

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

PAYLOAD:

```
{  
  "sub": "1234567890",  
  "name": "John Doe",  
  "iat": 1516239022  
}
```

VERIFY SIGNATURE

```
HMACSHA256(  
  base64UrlEncode(header) + "." +  
  base64UrlEncode(payload),  
  your-256-bit-secret  
) ☐ secret base64 encoded
```

 Signature Verified

SHARE JWT

Struktur JWT (...lanjutan)

bisa di-decode dengan mudah, tapi bukan berarti bisa “dibuka rahasianya”.

Decode vs Verify (ini yang sering bikin salah paham)

Decode → sangat mudah

Karena header dan payload itu cuma di-encode pakai **Base64**, bukan di-enkripsi. Jadi siapa pun bisa decode dan melihat isinya.

Verify (validasi) → tidak mudah

Bagian **signature** dibuat pakai secret key. Tanpa key ini, orang tidak bisa:

- Mengubah isi token
- Membuat token baru yang valid

Cara kerja JWT (Flow Authentication)

1. **User login** → kirim username & password ke server.
2. **Server verifikasi** → jika valid, server membuat JWT dan mengirim token ke client.
3. **Client simpan JWT (TOKEN)** → biasanya di localStorage atau cookie.
4. **Client request ke server** → setiap request API, JWT dikirim di **HTTP Header**:

```
Authorization: Bearer <token>
```

5. **Server verifikasi token** → jika token valid & belum expired, server izinkan akses.

Kelebihan JWT

- **Stateless** → server tidak perlu menyimpan session.
- Bisa digunakan di **microservices** atau sistem terdistribusi.
- Mudah dipakai di berbagai platform (mobile, web, IoT).

Kekurangan JWT

- Jika token bocor, siapa pun bisa akses (sampai expired).
- Token bisa menjadi **besar** (karena berisi data + signature).
- Tidak bisa “dicabut” dengan mudah sebelum expired, kecuali ada mekanisme **blacklist/refresh token**

Perbedaan JWT di Express VS Session

- **Stateless** = server tidak simpan data sesi; semua info ada di request (misalnya JWT).
 - **Stateful** = server simpan data sesi di memory/DB; client hanya bawa ID (misalnya PHP session).
-
- **JWT** → client simpan semua data user di token, server hanya verifikasi signature.
 - **Session** → server simpan data user di memory/database, client hanya pegang session ID (cookie).

Penerapan JWT Auth pada Express

untuk menerapkan JWT Auth pada Express install library dibawah ini:

```
npm install dotenv jsonwebtoken bcrypt
```

- dotenv digunakan untuk membaca environment variable dari file .env
- jsonwebtoken untuk menggunakan fungsi-fungsi JWT
- bcrypt untuk enkripsi

Demo membuat REST API dengan Raw MySql dan JWT

1) Struktur proyek

```
project-root/  
├─ package.json  
├─ .env          # (gitignored) DATABASE_URL, JWT_SECRET, PORT  
├─ .gitignore  
├─ config/  
│   ├─ db.js      # koneksi mysql2/promise  
│   └─ init.sql   # skrip create table  
├─ controllers/  
│   ├─ authController.js  
│   ├─ movieController.js  
│   └─ categoryController.js  
├─ middlewares/  
│   ├─ auth.js  
│   └─ validation.js # validasi untuk movie dan category  
├─ routes/  
│   ├─ authRoutes.js  
│   ├─ movieRoutes.js  
│   └─ categoryRoutes.js  
└─ server.js
```

2) package.json & instalasi

Kalau mulai dari nol , jalankan ini di terminal :

- `npm init -y`
- `npm install express mysql2 bcrypt jsonwebtoken dotenv`
- `npm install --save-dev nodemon`

Maka package.json akan otomatis terbuat dan berisi :

Keterangan :

- ◆ `express` → web framework
- ◆ `mysql2` → driver database MySQL (yang support promise)
- ◆ `bcrypt` → hash password
- ◆ `jsonwebtoken` → buat & verifikasi JWT
- ◆ `dotenv` → load `.env`
- ◆ `nodemon` (devDependencies) → auto-restart server

```
{} package.json > ...
1  {
2    "name": "latexpressrawmysqljwt",
3    "version": "1.0.0",
4    "main": "index.js",
5    "scripts": {
6      "test": "echo \"Error: no test specified\" && exit 1"
7    },
8    "keywords": [],
9    "author": "",
10   "license": "ISC",
11   "description": "",
12   "devDependencies": {
13     "nodemon": "^3.1.10"
14   },
15   "dependencies": {
16     "bcrypt": "^6.0.0",
17     "dotenv": "^17.2.1",
18     "express": "^5.1.0",
19     "jsonwebtoken": "^9.0.2",
20     "mysql2": "^3.14.3"
21   }
22 }
23
```

2) package.json & instalasi

Tambahkan pada scripts :

```
"scripts": {  
  "start": "node server.js",  
  "dev": "nodemon server.js"  
},
```

2) package.json & instalasi

Kalau buat package.json duluan seperti ini :

```
{
  "name": "movies-api",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "dependencies": {
    "bcrypt": "^5.1.0",
    "dotenv": "^16.0.0",
    "express": "^4.18.2",
    "jsonwebtoken": "^9.0.0",
    "mysql2": "^3.3.0"
  },
  "devDependencies": {
    "nodemon": "^2.0.22"
  }
}
```

Maka tinggal jalankan di terminal :

- `npm install`
- `npm install --save-dev nodemon`

3) .env (example) dan .gitignore

Taruh info sensitive / konfigurasi terpusat di environment :

```
⚙ .env
1  DATABASE_URL=mysql://root:passwordDBKita@localhost:3306/movies_db_rawMySQL_JWT
2  JWT_SECRET=secret_yang_kita_buat_sendiri_untuk_sign
3  PORT=3000
4
```

Taruh di .gitignore supaya tidak terupload di git karena berisi data sensitif

```
📁 .gitignore
1  node_modules
2  .env
3
```

3) .env (example) dan .gitignore

JWT_SECRET : Key rahasia yang kita tentukan sendiri untuk signature.

Kalau mau proteksi lebih (optional) , dapat dilakukan hashing lalu disimpan di .env pada JWT_SECRET.

4) config/init.sql — buat schema MySQL

```
config >  init.sql
1  -- create db + tables (sesuaikan nama db jika perlu)
2  CREATE DATABASE IF NOT EXISTS movies_db_rawMySQL_JWT;
3  USE movies_db_rawMySQL_JWT;
4
5  -- users
6  CREATE TABLE IF NOT EXISTS users (
7      id INT AUTO_INCREMENT PRIMARY KEY,
8      email VARCHAR(255) NOT NULL UNIQUE,
9      name VARCHAR(255),
10     password VARCHAR(255) NOT NULL,
11     createdAt DATETIME DEFAULT CURRENT_TIMESTAMP,
12     updatedAt DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
13 );
14
```

4) config/init.sql — buat schema MySQL

```
config > init.sql
14
15  -- categories
16  CREATE TABLE IF NOT EXISTS categories (
17      id INT AUTO_INCREMENT PRIMARY KEY,
18      name VARCHAR(255) NOT NULL,
19      description TEXT,
20      createdAt DATETIME DEFAULT CURRENT_TIMESTAMP,
21      updatedAt DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
22      userId INT,
23      CONSTRAINT fk_categories_user FOREIGN KEY (userId) REFERENCES users(id) ON DELETE SET NULL
24  );
25
26  -- movies
27  CREATE TABLE IF NOT EXISTS movies (
28      id INT AUTO_INCREMENT PRIMARY KEY,
29      title VARCHAR(255) NOT NULL,
30      year INT,
31      createdAt DATETIME DEFAULT CURRENT_TIMESTAMP,
32      updatedAt DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
33      categoryId INT,
34      userId INT,
35      CONSTRAINT fk_movies_category FOREIGN KEY (categoryId) REFERENCES categories(id) ON DELETE SET NULL,
36      CONSTRAINT fk_movies_user FOREIGN KEY (userId) REFERENCES users(id) ON DELETE SET NULL
37  );
38
```

4) config/init.sql — buat schema MySQL

Bisa dengan 3 CARA :

- Jalankan di terminal dengan: (Harus install mysql client)
`mysql -u root -p < config/init.sql`
- atau gunakan client MySQL GUI.
- atau gunakan cmd windows pembuatan db secara manual

5) config/db.js — koneksi db (mysql2/promise)

```
config > JS db.js > ...
1 // config/db.js
2 require('dotenv').config(); // load .env
3 const mysql = require('mysql2/promise'); // promise-based client
4
5 // parse DATABASE_URL like mysql://user:pass@host:port/dbname
6 const dbUrl = process.env.DATABASE_URL || '';
7 if (!dbUrl) {
8   throw new Error('DATABASE_URL belum di set di .env');
9 }
10 const url = new URL(dbUrl); // Node URL class
11
12 const pool = mysql.createPool({
13   host: url.hostname,           // host dari DATABASE_URL
14   user: url.username,           // user
15   password: url.password,       // password
16   database: url.pathname.replace(/^\//, ''), // path tanpa leading slash => dbname
17   port: url.port ? Number(url.port) : 3306,
18   waitForConnections: true,
19   connectionLimit: 10,
20   queueLimit: 0
21 });
22
23 module.exports = pool; // export pool untuk dipakai di controller
24
```

6) server.js — entry point

```
JS server.js > ...
1  // server.js
2  require('dotenv').config(); // load .env
3  const express = require('express');
4  const app = express();
5
6  // routes
7  const authRoutes = require('./routes/authRoutes');
8  const categoryRoutes = require('./routes/categoryRoutes');
9  const movieRoutes = require('./routes/movieRoutes');
10
11 // middleware global
12 app.use(express.json()); // parse JSON body
13
14 // mount routes
15 app.use('/api/auth', authRoutes);
16 app.use('/api/categories', categoryRoutes);
17 app.use('/api/movies', movieRoutes);
18
19 // basic error handler (simple)
20 app.use((err, req, res, next) => {
21   console.error(err);
22   res.status(err.status || 500).json({ error: err.message || 'Internal Server Error' });
23 });
24
25 // start
26 const PORT = process.env.PORT || 3000;
27 app.listen(PORT, () => {
28   console.log(`Server berjalan pada port ${PORT}`);
29 });
30
```

7) middlewares/auth.js — verifikasi JWT

```
middlewares > JS auth.js > ...
1 // middlewares/auth.js
2 const jwt = require('jsonwebtoken');
3 require('dotenv').config();
4
5 const jwtSecret = process.env.JWT_SECRET;
6 if (!jwtSecret) throw new Error('JWT_SECRET belum di set di .env');
7
8 module.exports = function auth(req, res, next) {
9   try {
10    // Ambil header Authorization: Bearer <token>
11    const authHeader = req.headers['authorization'];
12    if (!authHeader) return res.status(401).json({ message: 'No token provided' });
13
14    const parts = authHeader.split(' ');
15    if (parts.length !== 2 || parts[0] !== 'Bearer') {
16      return res.status(401).json({ message: 'Invalid Authorization header format' });
17    }
18
19    const token = parts[1];
20
21    // Verifikasi token
22    jwt.verify(token, jwtSecret, (err, decoded) => {
23      if (err) return res.status(401).json({ message: 'Invalid token' });
24      // Simpan payload user pada req.user untuk controller
25      req.user = { id: decoded.id, email: decoded.email };
26      next();
27    });
28  } catch (err) {
29    next(err);
30  }
31 };
32
```

7) middlewares/auth.js — verifikasi JWT

Perbandingan	req.headers[]	req.get()
Case-sensitive	✅ Ya	❌ Tidak
Return	Nilai mentah	Nilai header
Akses semua header	✅ Bisa	❌ Tidak
Cara yang disarankan di Express	⚠️ Tidak	✅ Ya
Contoh	req.headers['authorization']	req.get('Authorization')

`req.headers[]`

- Merupakan **akses langsung ke objek header mentah (raw object)**.
- Semua nama header **dalam huruf kecil (lowercase)**.

`req.get()`

- Fungsi **bawaan Express** yang lebih “rapi” untuk mengambil header.
- Nama header **tidak case-sensitive** — Express otomatis menormalkan.

8) middlewares/validation.js — validasi sederhana untuk movie & category

middlewares > JS validation.js > ...

```
1 // middlewares/validation.js
2 module.exports = {
3   validateCategoryCreate: (req, res, next) => {
4     const { name } = req.body;
5     const errors = [];
6     if (!name || typeof name !== 'string' || name.trim().length < 2) {
7       errors.push('name wajib (minimal 2 karakter)');
8     }
9     if (errors.length) return res.status(400).json({ errors });
10    next();
11  },
12
13  validateCategoryUpdate: (req, res, next) => {
14    // allow partial update; if name provided, validate
15    const { name } = req.body;
16    const errors = [];
17    if (name !== undefined && (typeof name !== 'string' || name.trim().length < 2)) {
18      errors.push('Jika disertakan, name minimal 2 karakter');
19    }
20    if (errors.length) return res.status(400).json({ errors });
21    next();
22  },
23 }
```


8) middlewares/validation.js — validasi sederhana untuk movie & category

```
middlewares > JS validation.js > ...
 2  module.exports = {
23
24    validateMovieCreate: (req, res, next) => {
25      const { title, year } = req.body;
26      const errors = [];
27      if (!title || typeof title !== 'string' || title.trim().length < 1) {
28        errors.push('title wajib');
29      }
30      if (year !== undefined && (!Number.isInteger(year) || year < 1800 || year > 3000)) {
31        errors.push('year harus integer valid (opsional)');
32      }
33      if (errors.length) return res.status(400).json({ errors });
34      next();
35    },
36
37    validateMovieUpdate: (req, res, next) => {
38      const { title, year } = req.body;
39      const errors = [];
40      if (title !== undefined && (typeof title !== 'string' || title.trim().length < 1)) {
41        errors.push('Jika disertakan, title tidak boleh kosong');
42      }
43      if (year !== undefined && (!Number.isInteger(year) || year < 1800 || year > 3000)) {
44        errors.push('Jika disertakan, year harus integer valid');
45      }
46      if (errors.length) return res.status(400).json({ errors });
47      next();
48    }
49  };
50
```

9) controllers/authController.js (register & login)

Hashing ≠ Encryption

Hashing

Satu arah (tidak bisa dibalik)

Untuk menyimpan password aman

bcrypt, SHA-256, Argon2

Encryption

Dua arah (bisa di-decrypt)

Untuk mengirim data rahasia

AES, RSA, DES

9) controllers/authController.js (register & login)

Apa itu *hashing*?

Hashing adalah proses mengubah data (misalnya password "123456") menjadi kode acak yang tetap panjangnya (misalnya \$2b\$10\$0qB7WL...) yang tidak bisa dibalik (irreversible).

Hashing adalah proses mengubah data (misalnya password) menjadi **kode unik** yang:

- Tidak bisa dikembalikan ke bentuk aslinya (irreversible).
- Tetap unik walau perbedaan kecil di input.
- Hasilnya selalu **panjang tetap** (misal 60 karakter).
- Cocok untuk **menyimpan password** secara aman di database.

Contoh:

Password: 123456

Setelah hashing: \$2b\$10\$0qB7WL2N2Zhb1AqWJARTo.gluOj9X1v6qcNjKGox78Xmawml.PqkC

9) controllers/authController.js (register & login)

Apa itu *salt* ?

📦 Contoh:

Input (plain)

Output (hash)

"123456"

\$2b\$10\$oqB7WL2N2Zhb1AqWJARTo.gIu0j9X1v6qcNjKGox78XmawmI.PqkC

"123456"

\$2b\$10\$wK4Yy7E3p3r/VwMoVnqKe0Z91h0wVYFq6g1c0HmPC1VvMu0Hj4Qp6

Perhatikan: input sama, hash-nya beda. Kenapa? Karena ada *salt*.

Salt adalah string acak yang ditambahkan sebelum atau selama proses hashing.

Fungsi salt:

- Biar hasil hash *unik* walaupun password-nya sama.
- Menghindari serangan *rainbow table* (kamus hash umum).

9) controllers/authController.js (register & login)

Apa itu bcrypt?

bcrypt adalah salah satu **algoritma hashing khusus untuk password**.



Kelebihan bcrypt:

- **Menggunakan salt otomatis.**
- **Memiliki saltRounds (biar hashing makin berat)** → memperlambat brute force.
- **Sudah battle-tested & recommended** di dunia industri (Node.js, PHP, Python, dsb).

9) controllers/authController.js (register & login)

⚙ Cara kerja bcrypt secara garis besar:

1. Kamu panggil:

```
js  
  
bcrypt.hash("123456", 10)
```

- "123456" = password asli
- 10 = saltRounds (semakin besar semakin lambat hashing, tapi lebih aman)

2. bcrypt:

- Buat salt acak
- Gabungkan password + salt
- Jalankan algoritma hashing
- Keluarkan hasil hash panjang

3. Saat login:

```
js  
  
bcrypt.compare("password_input", hash_di_database)
```

- bcrypt ambil salt dari hash di database
- hashing ulang input dengan salt itu
- bandingkan hasilnya
- return true atau false

9) controllers/authController.js (register & login)

```
controllers > JS authController.js > ...
1  // controllers/authController.js
2  const pool = require('../config/db');
3  const bcrypt = require('bcrypt');
4  const jwt = require('jsonwebtoken');
5  require('dotenv').config();
6
7  const jwtSecret = process.env.JWT_SECRET;
8  if (!jwtSecret) throw new Error('JWT_SECRET belum di set');
9
10 const SALT_ROUNDS = 10; // bcrypt salt rounds
11
```

9) controllers/authController.js (register & login)

```
controllers > JS authController.js > ...
11
12 module.exports = {
13   // POST /api/auth/register
14   register: async (req, res, next) => {
15     try {
16       const { email, password, name } = req.body;
17
18       // validasi sederhana
19       if (!email || !password) return res.status(400).json({ message: 'email & password wajib' });
20       if (password.length < 6) return res.status(400).json({ message: 'password minimal 6 karakter' });
21
22       // cek apakah user sudah ada
23       const [existing] = await pool.execute('SELECT id FROM users WHERE email = ?', [email]);
24       if (existing.length > 0) {
25         return res.status(409).json({ message: 'Email sudah terdaftar' });
26       }
27
28       // hash password
29       const hashed = await bcrypt.hash(password, SALT_ROUNDS);
30
31       // insert user
32       const [result] = await pool.execute(
33         'INSERT INTO users (email, name, password) VALUES (?, ?, ?)',
34         [email, name || null, hashed]
35       );
36
37       const insertedId = result.insertId;
38       return res.status(201).json({ id: insertedId, email, name: name || null });
39     } catch (err) {
40       next(err);
41     }
42   },
43 }
```


9) controllers/authController.js (register & login)

```
controllers > JS authController.js > ...
12  module.exports = {
43
44    // POST /api/auth/login
45    login: async (req, res, next) => {
46      try {
47        const { email, password } = req.body;
48        if (!email || !password) return res.status(400).json({ message: 'email & password wajib' });
49
50        // ambil user
51        const [rows] = await pool.execute('SELECT * FROM users WHERE email = ?', [email]);
52        if (rows.length === 0) return res.status(401).json({ message: 'Invalid credentials' });
53
54        const user = rows[0];
55
56        // bandingkan password
57        const match = await bcrypt.compare(password, user.password);
58        if (!match) return res.status(401).json({ message: 'Invalid credentials' });
59
60        // sign JWT (jangan simpan password di token)
61        const payload = { id: user.id, email: user.email };
62        const token = jwt.sign(payload, jwtSecret, { expiresIn: '1h' });
63
64        return res.json({
65          message: 'Login berhasil',
66          token,
67          user: { id: user.id, email: user.email, name: user.name }
68        });
69      } catch (err) {
70        next(err);
71      }
72    }
73  };
```

9) controllers/authController.js (register & login)

- Token yang diberikan ke user **hanya valid selama 1 jam**.
- Setelah lewat 1 jam → token **kadaluarsa (expired)**.
- Kalau token expired dan user akses endpoint yang butuh Authorization, maka akan ditolak (401 Unauthorized).
- Supaya bisa lanjut akses API, user harus **login lagi** (atau kalau ada mekanisme **refresh token**, cukup pakai refresh token untuk minta token baru tanpa login ulang).

JWT_SECRET VS JWT (TOKEN)

- `jwtSecret` → kunci rahasia, hanya server tahu, untuk membuat & memvalidasi token.
- JWT token → hasilnya (string panjang), dikirim ke client, lalu dipakai client untuk bukti login.

Analogi gampang:

- `jwtSecret` = stempel rahasia perusahaan (hanya bos punya).
- JWT token = surat yang sudah distempel. Orang lain bisa baca isi surat, tapi tidak bisa bikin surat palsu tanpa stempel aslinya.

1. jwtSecret (di .env)

Apa itu?

Kunci rahasia (string random) yang **server simpan** untuk *menandatangani* dan *memverifikasi* token.

Dimana disimpan?

- Di file .env (misalnya JWT_SECRET=mySecretKey123).
- **Hanya backend yang tahu**, jangan pernah dikirim ke frontend.

Gunanya?

Saat bikin token:

- `const token = jwt.sign(payload, process.env.JWT_SECRET, { expiresIn: '1h' });`
- → token ditandatangani pakai jwtSecret.

Saat verifikasi:

- `const decoded = jwt.verify(token, process.env.JWT_SECRET);`
- → server pastikan token valid dan tidak diubah.

2. JWT (hasil token ketika login)

Apa itu?

Token string (format Base64) yang dihasilkan setelah user login.

Isinya apa?

Biasanya 3 bagian (dipisah .):

- **Header** → info algoritma (alg) & tipe token (typ)
- **Payload** → data user (misalnya id, email)
- **Signature** → hasil enkripsi payload dengan jwtSecret

Dimana disimpan?

- Di browser client (localStorage, sessionStorage, cookie, memory).

Gunanya?

Client kirim token ini di Authorization: Bearer <token> setiap request → server cek dengan jwtSecret.

10) controllers/categoryController.js

```
controllers > JS categoryController.js > ...
1  // controllers/categoryController.js
2  const pool = require('../config/db');
3
4  module.exports = {
5    // GET /api/categories
6    getAll: async (req, res, next) => {
7      try {
8        const [rows] = await pool.execute('SELECT * FROM categories ORDER BY id DESC');
9        res.json(rows);
10     } catch (err) { next(err); }
11   },
12
13   // GET /api/categories/:id
14   getById: async (req, res, next) => {
15     try {
16       const id = parseInt(req.params.id, 10);
17       const [rows] = await pool.execute('SELECT * FROM categories WHERE id = ?', [id]);
18       if (rows.length === 0) return res.status(404).json({ message: 'Category tidak ditemukan' });
19       res.json(rows[0]);
20     } catch (err) { next(err); }
21   },
22 }
```

10) controllers/categoryController.js

controllers > JS categoryController.js > [?] <unknown> > create

```
4  module.exports = {
23  // POST /api/categories (protected)
24  create: async (req, res, next) => {
25    try {
26      const { name, description } = req.body;
27      const userId = req.user ? req.user.id : null; // siapa yang membuat (opsional)
28
29      const [result] = await pool.execute(
30        'INSERT INTO categories (name, description, userId) VALUES (?, ?, ?)',
31        [name, description || null, userId]
32      );
33      res.status(201).json({ id: result.insertId, name, description: description || null });
34    } catch (err) { next(err); }
35  },
36}
```

10) controllers/categoryController.js

```
controllers > JS categoryController.js > [?] <unknown> > [?] update
 4  module.exports = {
36
37  // PUT /api/categories/:id (protected)
38  update: async (req, res, next) => {
39    try {
40      const id = parseInt(req.params.id, 10);
41      const { name, description } = req.body;
42
43      // build dynamic update (simple)
44      const fields = [];
45      const values = [];
46      if (name !== undefined) { fields.push('name = ?'); values.push(name); }
47      if (description !== undefined) { fields.push('description = ?'); values.push(description); }
48
49      if (fields.length === 0) return res.status(400).json({ message: 'Nothing to update' });
50
51      values.push(id);
52      const sql = `UPDATE categories SET ${fields.join(', ')} WHERE id = ?`;
53      const [result] = await pool.execute(sql, values);
54
55      if (result.affectedRows === 0) return res.status(404).json({ message: 'Category tidak ditemukan' });
56      res.json({ message: 'Category diperbarui' });
57    } catch (err) { next(err); }
58  },
59 }
```


10) controllers/categoryController.js

controllers > JS categoryController.js > [?] <unknown> > remove

```
4  module.exports = {
60  // DELETE /api/categories/:id (protected)
61  remove: async (req, res, next) => {
62    try {
63      const id = parseInt(req.params.id, 10);
64      const [result] = await pool.execute('DELETE FROM categories WHERE id = ?', [id]);
65      if (result.affectedRows === 0) return res.status(404).json({ message: 'Category tidak ditemukan' });
66      res.json({ message: 'Category dihapus' });
67    } catch (err) { next(err); }
68  }
69 };
70
71
```

11) controllers/movieController.js

```
controllers > JS movieController.js > ...
1 // controllers/movieController.js
2 const pool = require('../config/db');
3
4 module.exports = {
5   // GET /api/movies
6   getAll: async (req, res, next) => {
7     try {
8       // join category to get category name
9       const [rows] = await pool.execute(
10         `SELECT m.*, c.name AS categoryName
11          FROM movies m
12          LEFT JOIN categories c ON m.categoryId = c.id
13          ORDER BY m.id DESC`
14       );
15       res.json(rows);
16     } catch (err) { next(err); }
17   },
18
19   // GET /api/movies/:id
20   getById: async (req, res, next) => {
21     try {
22       const id = parseInt(req.params.id, 10);
23       const [rows] = await pool.execute(
24         `SELECT m.*, c.name AS categoryName
25          FROM movies m
26          LEFT JOIN categories c ON m.categoryId = c.id
27          WHERE m.id = ?`, [id]
28       );
29       if (rows.length === 0) return res.status(404).json({ message: 'Movie tidak ditemukan' });
30       res.json(rows[0]);
31     } catch (err) { next(err); }
32   },
33 }
```

11) controllers/movieController.js

```
controllers > JS movieController.js > ...
4  module.exports = {
34  // POST /api/movies (protected)
35  create: async (req, res, next) => {
36    try {
37      const { title, year, categoryId } = req.body;
38      const userId = req.user ? req.user.id : null;
39
40      const [result] = await pool.execute(
41        'INSERT INTO movies (title, year, categoryId, userId) VALUES (?, ?, ?, ?)',
42        [title, year || null, categoryId || null, userId]
43      );
44      res.status(201).json({ id: result.insertId, title, year: year || null, categoryId: categoryId || null });
45    } catch (err) { next(err); }
46  },
47 }
```

11) controllers/movieController.js

```
controllers > JS movieController.js > ...
4  module.exports = {
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48  // PUT /api/movies/:id (protected)
49  update: async (req, res, next) => {
50    try {
51      const id = parseInt(req.params.id, 10);
52      const { title, year, categoryId } = req.body;
53
54      const fields = [];
55      const values = [];
56      if (title !== undefined) { fields.push('title = ?'); values.push(title); }
57      if (year !== undefined) { fields.push('year = ?'); values.push(year); }
58      if (categoryId !== undefined) { fields.push('categoryId = ?'); values.push(categoryId); }
59
60      if (fields.length === 0) return res.status(400).json({ message: 'Nothing to update' });
61
62      values.push(id);
63      const sql = `UPDATE movies SET ${fields.join(', ')} WHERE id = ?`;
64      const [result] = await pool.execute(sql, values);
65      if (result.affectedRows === 0) return res.status(404).json({ message: 'Movie tidak ditemukan' });
66      res.json({ message: 'Movie diperbarui' });
67    } catch (err) { next(err); }
68  },
69 }
```

11) controllers/movieController.js

```
controllers > JS movieController.js > ...
4  module.exports = {
5
70  // DELETE /api/movies/:id (protected)
71  remove: async (req, res, next) => {
72    try {
73      const id = parseInt(req.params.id, 10);
74      const [result] = await pool.execute('DELETE FROM movies WHERE id = ?', [id]);
75      if (result.affectedRows === 0) return res.status(404).json({ message: 'Movie tidak ditemukan' });
76      res.json({ message: 'Movie dihapus' });
77    } catch (err) { next(err); }
78  }
79 };
80
```

12) routes/*.js

```
routes > JS authRoutes.js > ...
1  const express = require('express');
2  const router = express.Router();
3  const authController = require('../controllers/authController');
4
5  router.post('/register', authController.register);
6  router.post('/login', authController.login);
7
8  module.exports = router;
9
10
```

12) routes/*.js

```
routes > JS categoryRoutes.js > ...
1  const express = require('express');
2  const router = express.Router();
3  const categoryController = require('../controllers/categoryController');
4  const auth = require('../middlewares/auth');
5  const { validateCategoryCreate, validateCategoryUpdate } = require('../middlewares/validation');
6
7  router.get('/', categoryController.getAll);           // public
8  router.get('/:id', categoryController.getById);      // public
9  router.post('/', auth, validateCategoryCreate, categoryController.create); // protected
10 router.put('/:id', auth, validateCategoryUpdate, categoryController.update); // protected
11 router.delete('/:id', auth, categoryController.remove); // protected
12
13 module.exports = router;
```

12) routes/*.js

```
routes > JS movieRoutes > ...
1  const express = require('express');
2  const router = express.Router();
3  const movieController = require('../controllers/movieController');
4  const auth = require('../middlewares/auth');
5  const { validateMovieCreate, validateMovieUpdate } = require('../middlewares/validation');
6
7  router.get('/', movieController.getAll);           // public
8  router.get('/:id', movieController.getById);       // public
9  router.post('/', auth, validateMovieCreate, movieController.create); // protected
10 router.put('/:id', auth, validateMovieUpdate, movieController.update); // protected
11 router.delete('/:id', auth, movieController.remove); // protected
12
13 module.exports = router;
14
```


Endpoint atau route hanya bisa diakses kalau user sudah login & membawa JWT valid.

Route public (tidak protected)

Semua orang bisa akses tanpa login:

```
// routes/authRoutes.js
router.post("/register", register); // siapa saja bisa daftar
router.post("/login", login);      // siapa saja bisa login
```

Route protected (pakai middleware auth)

Hanya user yang sudah login dan membawa token valid:

```
// routes/movieRoutes.js
const auth = require("../middlewares/auth");

router.get("/", auth, movieController.getAll); // protected
router.post("/", auth, movieController.create); // protected
```

13) Alur aplikasi (flow)

Client --> |HTTP request| Express_Router

Express_Router --> |route| Validation_Middleware

Validation_Middleware --> Auth_Middleware{protected?}

Auth_Middleware --> |ok| Controller

Validation_Middleware --> |ok| Controller

Controller --> |query| MySQL_DB[(MySQL Pool)]

MySQL_DB --> Controller

Controller --> |JSON response| Client

Penjelasan singkat:

Client → Express Router → Validation middleware (cek body) → jika protected → auth.js (cek JWT) → controller → DB (pool) → controller mengembalikan JSON ke client.

14) Pengujian (contoh dengan **Thunder Client**)

1. Pastikan Database dan tabel-tabelnya sudah terbuat dan terhubung (kalau belum, jalankan config/init.sql)
2. Jalankan server npm run dev di terminal.

14) Pengujian (contoh dengan Thunder Client)

1. Register user

- Method: `POST`
- URL: `http://localhost:3000/api/auth/register`
- Body (JSON):

json

```
{  
  "email": "cika@example.com",  
  "password": "secret123",  
  "name": "Cika"  
}
```

- Expected: `201` JSON `{ id, email, name }` atau `409` jika email sudah ada.

14) Pengujian (contoh dengan Thunder Client)

2. Login

- Method: `POST`
- URL: `http://localhost:3000/api/auth/login`
- Body:

```
json
{
  "email": "cika@example.com",
  "password": "secret123"
}
```

- Expected: `200` JSON:

```
json
{
  "message": "Login berhasil",
  "token": "<JWT token>",
  "user": { "id": 1, "email": "cika@example.com", "name": "Cika" }
}
```

- Copy `token` untuk Authorization header.

- Beda user akan mendapatkan token yang berbeda saat login.
- Simpan info TOKEN dari LOGIN sebelum user tsb mengakses ROUTE PROTECTED.

14) Pengujian (contoh dengan Thunder Client)

The screenshot displays the Thunder Client interface. On the left, the 'Body' tab is selected, showing a JSON request with email and password. On the right, the 'Response' tab shows a 200 OK status with a JSON response containing a message, a token (highlighted with a red box), and user information.

Request:

```
POST http://localhost:3000/api/auth/login
{
  "email": "Ana@example.com",
  "password": "secret123"
}
```

Response:

```
{
  "message": "Login berhasil",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MiwiZW1haWwiOiJBbmFAZXhhbXBsZS5jb20iLCJpYXQiOiE3NTYwMzE1NzEsImV4cCI6MTc1NjAzNTE3MX0.eyJ0PT9rY67VRIVjWNSxEbVw",
  "user": {
    "id": 2,
    "email": "Ana@example.com",
    "name": "Ana"
  }
}
```

- Beda user akan mendapatkan token yang berbeda saat login.
- Simpan info TOKEN dari LOGIN sebelum user tsb mengakses ROUTE PROTECTED.

14) Pengujian (contoh dengan Thunder Client)

3. Buat Category (protected)

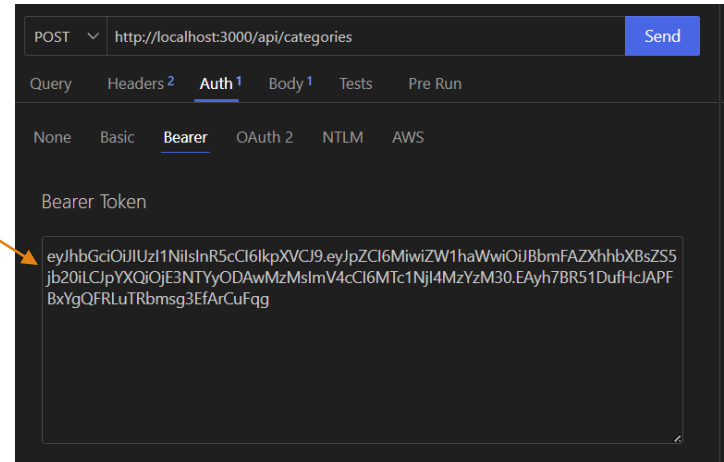
- Method: `POST`
- URL: `http://localhost:3000/api/categories`
- Header: `Authorization: Bearer <token>`
- Body:

```
json

{
  "name": "Sci-Fi",
  "description": "Science fiction movies"
}
```

- Expected: `201` JSON `{ id, name, description }`

Masukkan JWT (TOKEN) yang didapatkan dari proses Login



14) Pengujian (contoh dengan Thunder Client)

4. Get All Categories

- Method: GET `http://localhost:3000/api/categories`
- Expected: 200 list categories.

5. Update Category

- Method: PUT `http://localhost:3000/api/categories/1`
- Header: Authorization: Bearer <token>
- Body:

json

```
{ "description": "Deskripsi baru" }
```

- Expected: 200 { message: 'Category diperbarui' }

6. Delete Category

- Method: DELETE `http://localhost:3000/api/categories/1`
- Header: Authorization: Bearer <token>
- Expected: 200 { message: 'Category dihapus' }

14) Pengujian (contoh dengan Thunder Client)

7. Buat Movie (protected)

- Method: `POST` `http://localhost:3000/api/movies`
- Header: `Authorization: Bearer <token>`
- Body:

json

```
{  
  "title": "Inception",  
  "year": 2010,  
  "categoryId": 1  
}
```

- Expected: `201 { id, title, year, categoryId }`

14) Pengujian (contoh dengan Thunder Client)

8. Get All Movies

- Method: `GET` `http://localhost:3000/api/movies`
- Expected: `200` list movies dengan field `categoryName` jika ada category.

9. Update Movie

- Method: `PUT` `http://localhost:3000/api/movies/1`
- Header: `Authorization: Bearer <token>`
- Body:

json

```
{ "title": "Inception (updated)" }
```

10. Delete Movie

- Method: `DELETE` `http://localhost:3000/api/movies/1`
- Header: `Authorization: Bearer <token>`

Contoh Login

The screenshot displays a REST client interface with a POST request to `http://localhost:3000/api/auth/login`. The request body is a JSON object containing email and password. The response is a 200 OK status with a JSON body containing a success message, a token, and user details. The token is highlighted with an orange circle.

Request:

```
POST http://localhost:3000/api/auth/login
{
  "email": "Ana@example.com",
  "password": "secret123"
}
```

Response:

```
{
  "message": "Login berhasil",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MiwiZW1haWwiOiJBbmFAZXhhbXBsZS5jb20iLCJpYXQiOiJlMzNlYmZlMTNzEsImV4cCI6MTc1NjAzNTE3MX0.ngSDHUMyyQj67hho02A-Uj0PT0cY67VRIVJWNSxEbVw",
  "user": {
    "id": 2,
    "email": "Ana@example.com",
    "name": "Ana"
  }
}
```

JWT

Misal user berhasil login, server balas:

```
{  
  "message": "Login successful",  
  "token":  
  "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwiZW1haWwiOiJjaWthQGV4YW1wbGUuY29tliwiaWF0IjoxNzI0NDc4NzY1LCJleHAiOjE3MjQ0ODIzNjV9.RkSkF0H2sfbbAjSx8FqZ_VhYQXe1Lg7kJCNk8UpLmU8"  
}
```

Token ini terdiri dari 3 bagian (dipisahkan .):

1 Header (algoritma & tipe)

json

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

- `alg` → algoritma enkripsi (`HS256` = HMAC SHA256)
- `typ` → jenis token (JWT)

2 Payload (data / claims)

json

```
{
  "id": 1,
  "email": "cika@example.com",
  "iat": 1724478765,
  "exp": 1724482365
}
```

- `id` → ID user dari database
- `email` → email user
- `iat` → Issued At (waktu token dibuat, dalam UNIX timestamp)
- `exp` → Expired At (token habis 1 jam kemudian, dalam UNIX timestamp)

Jadi kalau di Thunder Client kamu **copy-paste token** itu ke Authorization
→ Bearer Token, maka request protected route ke `/api/movies` atau
`/api/categories` akan sukses

3 Signature (validasi)

nginx

```
RKSkF0H2sfbbAjsx8FqZ_VhYQXe1Lg7kJCNk8UpLmU8
```

Ini hasil hash dari `(header + payload + secret)`.

Gunanya supaya token tidak bisa dimodifikasi sembarangan.

Tambahan - Cors

1. Frontend & Backend 1 domain / 1 port → Tidak perlu CORS karena origin-nya sama (domain, protokol, port sama).
2. Frontend & Backend beda port (development) → Origin berbeda (beda port → beda origin), maka browser akan block request tanpa CORS. Jadi perlu tambahkan cors middleware di backend.

- npm install cors

server.js :

```
js

const cors = require('cors');

// izinkan semua origin (untuk dev)
app.use(cors());

// atau lebih aman: hanya frontend tertentu
app.use(cors({
  origin: 'http://localhost:5173',
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization']
}));
```

3. Frontend & Backend beda domain (production) → Harus pakai CORS, kalau tidak request frontend akan ditolak browser. Set cors({ origin: 'https://frontend.com' }).

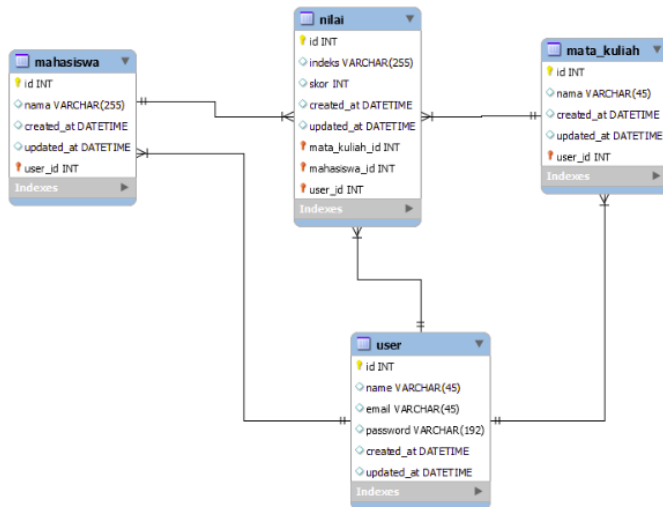
Kesimpulan

- JWT
- Penerapan JWT Auth di Express.js

TUGAS

kerjakan soal di bawah ini didalam folder **tugas-rest-api-rawMySQL-jwt**

buatlah CRUD REST API, auth login, register, dan middleware jwt auth berdasarkan skema database dibawah ini



buat 3 CRUD berbeda sesuai dengan nama tabel masing-masing, untuk POST tabel nilai yang diperbolehkan di input dalam body json hanya **mata_kuliah_id**, **mahasiswa_id** dan **nilai** lalu **nilai** hanya bisa diinput dari 0 s/d 100, jika diisi dibawah 0 atau diatas 100 maka akan ada pesan data nilai salah untuk indeks didapatkan dari function yang ada dalam kode backend dengan ketentuan

```
nilai >= 80 indeksnya A,
nilai >= 70 dan nilai < 80 indeksnya B,
nilai >= 60 dan nilai < 70 indeksnya C,
nilai >= 50 dan nilai < 60 indeksnya D,
nilai < 50 indeksnya E
```

Terima Kasih

